



Department of Computer Science and Engineering
GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

Mini Project Report on

**WIRELESS VOLUME CONTROL BY HAND
DETECTION**

Submitted in partial fulfilment of the requirements for the award of the

Bachelor of Technology

In

Department of Computer Science and Engineering

By

1.Kunta Sreeshma	20241A05Q9
2.Maddhuri Sowmya Sree	20241A05R2
3.Nenavath Akhila	20241A05R7
4.S.Navika	20241A05S3
5.Sai Naveena Kode	20241A05S4

Under the Esteemed guidance of

Dr.P.Chandra Sekhar Reddy

Professor



GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

CERTIFICATE

This is to certify that the mini project entitled “**Wireless Volume Control By Hand Detection**” is submitted by **Kunta Sreeshma(20241A05Q9)**, **Maddhuri Sowmya Sree(20241A05R2)**, **Nenavath Akhila (20241A05R7)**, **Navika(20241A05S3)**, **Sai Naveena Kode(20241A05S4)**, In partial fulfilment of the award of degree in BACHELOR OF TECHNOLOGY in Computer Science and Engineering during academic year 2022-2023.

INTERNAL GUIDE

Dr.P.Chandra Sekhar Reddy
Professor

HEAD OF THE DEPARTMENT

Dr.K.Madhavi
Professor

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

There are many people who helped us directly and indirectly to complete our project successfully. We would like to take this opportunity to thank one and all. First we wish to express our deep gratitude towards our internal guide **Dr.P.Chandra Sekhar Reddy, Professor**, Department of CSE for her/his support in the completion of our project report. We wish to express our honest and sincere thanks to **Dr. K. Madhavi, HOD, Department of CSE** and to our principal **Dr. J. Praveen** for providing the facilities to complete mini project. We would like to thank all our faculty and friends for their help and constructive criticism during the project completion phase. Finally, we are very much indebted to our parents for their moral support and encouragement to achieve goals.

Kunta Sreeshma-20241A05Q9

Maddhuri Sowmya Sree 20241A05R2

Nenavath Akhila-20241A05R7

S Navika-20241A05S3

Sai Naveena Kode-20241A05S4

DECLARATION

We hereby declare that the mini project entitled “**Wireless Volume Control By Hand Detection**” is the work done during the period from **12th Jan 2023 to 3rd June 2023** and is submitted in the partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering from Gokaraju Rangaraju Institute of Engineering and Technology (Autonomous under Jawaharlal Nehru Technology University, Hyderabad).The results embodied in this project have not been submitted to any other university or Institution for the award of any degree or diploma.

Kunta Sreeshma-20241A05Q9

Maddhuri Sowmya Sree 20241A05R2

Nenavath Akhila-20241A05R7

S Navika-20241A05S3

Sai Naveena Kode-20241A05S4

	Table of Contents	
Chapter	TITLE	Page No
	Abstract (150-250 Words)	1
I	Introduction	2-3
	1.1 Existing System	
	1.1.1 Limitations in Existing System	
	1.2 Proposed System	
	1.2.1 Advantages over Existing System	
II	Literature Survey	4-5
III	Software Requirement Specifications	6-14
	3.1 IEEE Standard 1. Introduction 1.1 Purpose of the requirements document 1.2 Scope of the product 1.3 Definitions, acronyms and abbreviations 1.4 References 1.5 Overview of the remainder of the document 2. General description 2.1 Product perspective 2.2 Product functions 2.3 User characteristics 2.4 General constraints 2.5 Assumptions and dependencies 3. Specific requirements , covering functional, non-functional and interface requirements. This is obviously the most substantial part of the document but because of the wide variability in organizational practice, it is not appropriate to define a standard structure for this section. The requirements may document external interfaces, describe system functionality and performance, and specify logical database requirements, design constraints, emergent system properties and quality characteristics.	
	3.2 Feasibility Study (No Text Book Description)	
	3.2.1 Technical Feasibility (100 to 200 Words)	
	3.2.2 Economic Feasibility (100 to 200 Words)	
	3.2.3 Operational Feasibility (100 to 200 Words)	
IV	Design (No Textbook Definitions)	15-20
	4.1 Project Description	

	4.1.1 System Use case Diagrams (In detailed Explanation)																									
	4.1.2 Class Diagrams (In detailed Explanation)																									
	4.1.3 Data Flow Diagram (DFD) (In detailed Explanation) Any other UML Diagrams																									
V	Implementation	21-41																								
	5.1 Source code (Main) 5.2 Screen shots (Results)																									
VI	Testing	42-43																								
VII	Conclusion& Future Scope (Note: 150 to 250 words)	44-46																								
VIII	References (Papers, Websites, Textbooks)	47																								
IX	List of Figures <table border="1"> <thead> <tr> <th>Figure No</th><th>Figure Title</th><th>Page No</th></tr> </thead> <tbody> <tr> <td>a</td><td>Project Architecture</td><td>15</td></tr> <tr> <td>b</td><td>Overall Usecase Diagram</td><td>16</td></tr> <tr> <td>c</td><td>Usecase for Hand Tracking</td><td>17</td></tr> <tr> <td>d</td><td>Usecase for Hand Tracing</td><td>17</td></tr> <tr> <td>e</td><td>Usecase for Volume Controller</td><td>18</td></tr> <tr> <td>f</td><td>Class diagram</td><td>18</td></tr> <tr> <td>g</td><td>Data Flow diagram</td><td>19</td></tr> </tbody> </table> List of Tables Testing table-page no:42-43	Figure No	Figure Title	Page No	a	Project Architecture	15	b	Overall Usecase Diagram	16	c	Usecase for Hand Tracking	17	d	Usecase for Hand Tracing	17	e	Usecase for Volume Controller	18	f	Class diagram	18	g	Data Flow diagram	19	
Figure No	Figure Title	Page No																								
a	Project Architecture	15																								
b	Overall Usecase Diagram	16																								
c	Usecase for Hand Tracking	17																								
d	Usecase for Hand Tracing	17																								
e	Usecase for Volume Controller	18																								
f	Class diagram	18																								
g	Data Flow diagram	19																								

ABSTRACT:

Gesture recognition helps computers to understand mortal body language. This helps to make a more potent link between humans and machines, rather than just the introductory textbook stoner interfaces or graphical stoner interfaces(GUIs). With this in mind, an idea of volume regulator used hand gesture discovery has been proposed. For hand gesture recognition, the mortal body's movements are read by computer camera. The computer also makes use of this data as input to handle operations. The system is designed to produce an interface which will capture mortal hand gesture stoutly and will control the volume position by using computer vision ways. For this, Deep Learning ways similar as CNN have been appertained. After the hand discovery in captured frames, the coming step is to control the system volume depending on direction of hand movement. Our idea is also to create a advance volume controller for better efficiency in controlling . Our motive is to achieve effective volume controlling system for real time purposes.

I. Introduction:

1.1 Existing System:

Hand gesture recognition method using Convolutional Neural Networks(CNNs). The procedure involves the application of morphological filters, contour generation, polygonal approximation, and segmentation during preprocessing, in which they contribute to a better feature extraction. Training and testing are performed with different convolutional neural networks, compared with architectures known in the literature and with other known methodologies. All calculated metrics and convergence graphs obtained during training are analyzed and discussed to validate the robustness of the proposed method.

1.1.1 Limitations of Existing System:

- It requires more resources such as computing power, memory and time to process the huge input data.
- The existing system only provides limited interaction options such as hand detection or gesture recognition where as our project integrates with it and forms a functionality of system.
- Existing system have has lower accuracy in detecting hands in real time as compared to proposed system because it considers many factors, while proposed one is more specific and fast and algorithms used can be tuned for higher accuracy.

1.2 Proposed System:

We are developing a system which controls the volume by detecting human hand gestures through visual based interface of device. The algorithms used in our system are computer vision and CNN.-the IDE used for the development of this system is PyCharm where python language is used. OpenCV of version 3.x is which provides wide range of functions for image and video processing-in this system prominently index and thumb finger are detected and distance between them is Calculated accordingly and volume is either increased or decreased. The volume bar is created at top right corner to indicate volume fluctuation.

1.2.1 Advantages over Existing system:

- Enhanced Volume Control: The proposed system utilizes hand gesture recognition to control the volume. This provides a more intuitive and natural way for users to adjust the volume compared to traditional methods. Instead of using physical buttons or sliders, users can simply

use hand gestures, making the volume control process more engaging and interactive.

- **Real-time Interaction:** The proposed system is designed for real-time purposes, allowing users to dynamically control the volume level as they perform hand gestures. This enables instant adjustments without any noticeable delay, enhancing the user experience and convenience.
- **Computer Vision Techniques:** The proposed system leverages computer vision techniques, specifically using OpenCV and CNN algorithms, to detect and track hand gestures. By analyzing the movements and positions of the fingers, the system accurately determines the intended volume adjustment. This approach improves the accuracy and reliability of gesture recognition compared to other methods that rely solely on morphological filters and contour generation.
- **Flexible Gestures:** The system focuses on detecting the index and thumb fingers and calculating the distance between them to determine the volume adjustment. This approach allows for a wide range of gestures that can be recognized, providing flexibility for users to choose the hand gesture that feels most comfortable to them. Users can easily adapt their gestures based on their preferences or physical limitations.
- **Visual Feedback:** The volume bar displayed at the top right corner of the interface provides visual feedback to indicate the current volume level. This visual representation enhances user understanding and awareness of the volume changes, enabling them to make more precise adjustments according to their preferences.
- **Development Environment:** The system is developed using PyCharm as the Integrated Development Environment (IDE) and the Python programming language. PyCharm offers a robust development environment with features like code completion, debugging tools, and easy integration with libraries such as OpenCV. Python is widely used in the field of computer vision and provides a rich ecosystem of libraries and resources, making development efficient and accessible.

1.3 Definitions, Acronyms and Abbreviations:

- OpenCV: It was created to give computer vision applications a uniform framework and to speed up the incorporation of Artificial Intelligence into products.
- MediaPipe: This open-source framework enables the construction of pipelines for CV inference across a variety of sensory data, including audio and video. Such a perception pipeline can be created as a graph of modular parts using MediaPipe.

II. Literature Survey:

1. Real-time finger tracking and contour detection for gesture recognition using OpenCV

Authors: Ruchi Manish Gurav, Premanand K. Kadbe

The paper titled "Real-time finger tracking and contour detection for gesture recognition using OpenCV" proposes a method for accurately tracking fingers and detecting hand contours in real-time. The authors utilize OpenCV's computer vision capabilities and present a step-by-step approach involving background subtraction, noise reduction, hand region segmentation, finger candidate extraction, and finger tracking. Experimental results demonstrate the effectiveness of the method in real-time scenarios, showcasing its potential for gesture recognition applications.

2. Hand Gesture Recognition Techniques for Human-Computer Interaction

Authors: Meenakshi Panwar, Pawan Singh Mehra

The paper titled "Hand Gesture Recognition Techniques for Human-Computer Interaction" presents an overview of various techniques used in hand gesture recognition for HCI. It discusses traditional methods based on color, shape, and motion features, as well as advanced approaches utilizing machine learning algorithms. The paper also explores applications of hand gesture recognition in domains such as gaming, sign language interpretation, virtual reality, and robotics. It serves as a valuable resource for researchers and practitioners interested in this field.

3. Hand Gesture Recognition using OpenCV and Python

Authors: Surya Narayan Sharma, Dr. A Rengarajan

The paper titled "Hand Gesture Recognition using OpenCV and Python" presents an approach for real-time hand gesture recognition using OpenCV and Python. The authors cover the steps involved in hand detection, feature extraction, and

classification. Experimental results demonstrate the effectiveness of the proposed system, showcasing its potential for enhancing human-computer interaction.

4. Volume Control using Gestures

Authors: Martendra Pratap Singh, Arzoo Poswal, Eshu Yadav

The paper titled "Volume Control using Gestures" proposes a system that allows volume adjustment through hand gestures. It highlights the significance of gesture recognition in human-computer interaction and discusses the implementation of the system using computer vision techniques. The experimental results demonstrate the effectiveness of using hand gestures for controlling volume levels.

III. Software Requirement Specifications:

3.1.1 Introduction:

3.1.1.1 Purpose of the requirements document:

The purpose of this document is to define the requirements for a wireless volume control system that utilizes hand detection technology to adjust the volume of an audio device. The system aims to provide users with a convenient and intuitive method to control the volume without the need for physical buttons or remote controls.

3.1.1.2 Scope of the product:

The wireless volume control system will consist of a hand detection sensor, a wireless communication module, and a volume control mechanism. It will allow users to control the volume of an audio device by detecting their hand gestures. The system will be designed to accurately detect the presence of a hand, track its position and movement, and recognize predefined hand gestures for volume control. It will establish a wireless connection with the audio device and send volume control commands based on the detected gestures. The system will also provide visual feedback to the user regarding the recognized gestures and the current volume level.

3.1.1.3 Definitions, acronyms and abbreviations:

- OpenCV: Open Source Computer Vision Library is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products
- MediaPipe: It is an open-source framework for building pipelines to perform computer vision inference over arbitrary sensory data such as video or audio. Using MediaPipe, such a perception pipeline can be built as a graph of modular components.
- Pykaw: It is a Python library that provides a high-level interface for controlling audio devices on the Windows operating system. The name "pykaw" stands for "Python Core Audio Windows" and reflects its purpose of allowing developers to interact with the core audio functionality in Windows.

- Hand tracking: It refers to the process of detecting and tracking the movements and positions of human hands in real-time using computer vision or sensor-based technologies. It involves capturing the motion and shape of hands, recognizing and tracking individual fingers, and extracting relevant information from hand movements.

Acronyms and Abbreviations:

- OpenCV: Open source computer vision
- FPS: Frame rate Per Second
- CNN: Convolutional Neural Networks

3.1.1.4 Overview of the remainder of the document:

Traditional volume control methods involve physical buttons, knobs, or remote controls, which may be inconvenient or require manual manipulation. The wireless volume control system using hand detection offers an innovative and hands-free approach to volume adjustment. By utilizing hand detection technology, the system can accurately detect and track the user's hand, recognize predefined gestures, and translate them into volume control commands. This enables users to adjust the volume simply by performing specific hand gestures, providing a seamless and natural user experience.

The system aims to enhance user convenience, accessibility, and user experience by eliminating the need for physical interactions with the audio device. It enables users to control the volume from a distance, allowing them to adjust the audio output without directly accessing the device. The wireless communication capabilities ensure flexibility and ease of use, enabling users to control the volume from various locations within the system's range,

The following sections of this document will detail the functional and non-functional requirements of the wireless volume control system, as well as the user interface requirements, system constraints, and system architecture. These requirements will serve as a guideline for the design, development, and implementation of the system to ensure its effectiveness, accuracy, responsiveness, and compatibility with a wide range of audio devices

3.1.2 GENERAL DESCRIPTION

3.1.2.1 Product Perspective:

The wireless volume control system will function as a standalone application or as an integrated feature within existing wireless devices. It will interact with the device's operating system or audio control mechanisms to adjust the volume levels.

3.1.2.2. Product Functions

- **Hand Gesture Recognition:** The system should accurately recognize hand gestures performed by the user.
- **Volume Control:** The system should enable users to control the volume by performing specific hand gestures.
- **Multiple Gestures:** The system should support different gestures for various volume functions like increasing, decreasing, muting, or adjusting volume levels.
- **Real-time Responsiveness:** The system should provide immediate feedback and adjust the volume in real-time.
- **Sensitivity Adjustment:** The system should allow users to customize the gesture recognition sensitivity based on their preferences and environmental conditions.
- **Compatibility:** The system should be compatible with a wide range of wireless devices and operating systems.

3.1.2.3 User Classes and Characteristics:

The wireless volume control system is designed for general users who prefer a hands-free and intuitive method of controlling the volume on their wireless devices. Users should be comfortable with performing hand gestures.

3.1.2.4 General description:

- **Accuracy:** Hand detection algorithms need to accurately identify and track hand movements in real-time. The system must have a low error rate to ensure reliable volume control. Inaccurate hand detection may result in unintended volume changes or lack of responsiveness.

- **Lighting Conditions:** The hand detection system may be sensitive to lighting conditions. It should be able to operate effectively under different lighting environments, such as bright sunlight or dimly lit rooms. Adequate lighting must be ensured for accurate hand detection and tracking.
- **Occlusion:** Hands can be partially or completely occluded by objects, clothing, or other body parts. The hand detection system should be robust enough to handle occlusions and still detect and track the hand accurately. Otherwise, it may result in inconsistent volume control or false detections.
- **Hand Variations:** People have different hand shapes, sizes, and appearances. The hand detection system should be able to handle these variations and detect hands reliably across different users. It should not be limited to specific hand shapes or sizes, ensuring a wider range of users can use the volume control feature.
- **Latency:** The system should respond quickly to hand movements to provide a seamless volume control experience. Any significant delay between hand movements and volume adjustments can lead to a poor user experience. Minimizing latency is crucial to ensure real-time and responsive volume control.
- **Robustness to False Positives/Negatives:** The hand detection system should avoid false positives (detecting hands when there are none) and false negatives (failing to detect hands when they are present). False positives may result in unintended volume changes, while false negatives would render the system non-functional.
- **User Adaptation:** The system should adapt to different users and their specific hand movements. It should be able to learn and adjust to individual hand gestures, allowing users to customize their preferred gestures for volume control. This adaptability will enhance user satisfaction and usability.
- **Gesture Differentiation:** The hand detection system needs to differentiate between volume control gestures and other hand movements or gestures. It should be able to accurately interpret the intended volume control gestures and ignore unintended gestures, such as waving or pointing.
- **Power Consumption:** If the hand detection system relies on sensors or cameras, power consumption should be considered. The system should be energy-efficient to minimize battery drain in portable devices or ensure sustainability in other applications.

3.1.2.5. Assumptions and Dependencies:

- The wireless devices used will have the necessary sensors or capabilities for hand detection and gesture recognition.
- The wireless devices will have an operating system that allows integration with the volume control system.
- The wireless devices will have appropriate connectivity (e.g., Wi-Fi, Bluetooth) for communication with the volume control system.

3.1.3 SPECIFIC REQUIREMENTS:

→ Functional Requirements:

- **Gesture Recognition:** The system should be able to recognize hand gestures accurately and reliably.
- **Volume Control:** The system should allow users to control the volume of a wireless device by performing specific hand gestures.
- **Multiple Gestures:** The system should support different hand gestures for controlling various volume functions such as increasing, decreasing, muting, or adjusting volume levels.
- **Sensitivity Adjustment:** The system should provide a mechanism to adjust the sensitivity of the gesture recognition to accommodate different users and environments.
- **Compatibility:** The system should be compatible with a wide range of wireless devices such as smartphones, tablets, laptops, or smart speakers.
- **Real-time Responsiveness:** The system should respond to hand gestures in real-time, providing immediate volume adjustments.
- **Calibration:** The system should offer a calibration process to ensure accurate gesture recognition and volume control.
- **Error Handling:** The system should handle errors gracefully and provide feedback to the user when a gesture is not recognized or volume control fails.

→ Non-functional Requirements:

- **Reliability:** The system should be highly reliable, consistently recognizing gestures and accurately controlling the volume.

- **Accuracy:** The gesture recognition system should have a high level of accuracy, minimizing false positives and false negatives.
- **Low Latency:** The system should have minimal latency, ensuring that there is minimal delay between performing a gesture and the volume adjustment.
- **User-Friendly:** The system should be easy to use and intuitive, allowing users to control the volume effortlessly using hand gestures.
- **Scalability:** The system should be scalable, supporting a large number of users and devices simultaneously.
- **Security:** The wireless volume control system should incorporate appropriate security measures to prevent unauthorized access or tampering.
- **Power Efficiency:** The system should be designed to minimize power consumption, maximizing the battery life of the wireless device.
- **Robustness:** The system should be resilient to environmental factors such as varying lighting conditions, background interference, or hand variations.
- **Accessibility:** The system should be accessible to users with different physical abilities and should accommodate different hand sizes or mobility limitations.
- **Compatibility:** The system should work seamlessly with various operating systems and wireless technologies commonly used in devices.

→ PERFORMANCE REQUIREMENTS:

Real-Time Processing:

- The hand detection and gesture recognition algorithms should operate in real-time, providing immediate response to the user's gestures.
- The system should achieve an acceptable frame rate to ensure smooth and responsive volume control.

Recognition Accuracy:

- The gesture recognition should have a high accuracy rate, minimizing false positives and false negatives, The system should be able to differentiate between different gestures with a high level of precision

Latency:

- The overall system latency, including hand detection, gesture recognition, and wireless communication, should be minimal.

- The time between performing a gesture and the corresponding volume adjustment should be imperceptible to the user.

→ **USABILITY REQUIREMENTS:**

User Interface:

The system should have a clear and intuitive user interface, providing visual feedback on recognized gestures and the current volume level. The user interface should be easy to understand and navigate, ensuring a user-friendly experience.

Gesture Training and Customization:

- The system should allow users to train and customize gestures based on their preferences.
- Users should be able to define and assign specific gestures to different volume control commands.

→ **ENVIRONMENTAL REQUIREMENTS:**

Lighting Conditions:

- The system should perform adequately under various lighting conditions, including lowlight or high-contrast environments.
- The hand detection and recognition algorithms should be robust enough to handle lighting variations.

→ **COMPATIBILITY REQUIREMENTS:**

Hardware Compatibility:

- The system should be compatible with a wide range of cameras or sensors capable of capturing video input.
- The wireless communication module should be compatible with the target audio devices, such as speakers or amplifiers

Software Compatibility:

- The system should be compatible with common operating systems, ensuring its usability across different platforms.

3.2 FEASIBILITY STUDY:

3.2.1 Technical Feasibility:

- **Hand detection and recognition technology:** The availability of reliable computer vision algorithms and machine learning techniques for hand detection and recognition is a key technical requirement. This technology has been widely researched and implemented successfully in various applications, indicating technical feasibility.
- **Hardware requirements:** The feasibility depends on the availability of suitable hardware components, such as cameras or sensors, with sufficient resolution and frame rate capabilities. The market offers a range of options that meet the requirements, making technical feasibility achievable.
- **Wireless communication:** The availability of wireless communication technologies, such as Bluetooth or Wi-Fi, is crucial for establishing a connection between the hand detection module and the audio device. These technologies are well-established and widely supported, ensuring technical feasibility.

3.2.2 Economic Feasibility:

- **Cost of development:** The feasibility study should consider the cost associated with developing the system, including software development, algorithm training, and integration with hardware components. The availability of resources and budget will determine the economic feasibility of the project.
- **Market demand:** Assessing the potential market demand for a volume control system using hand detection is essential. Market research can provide insights into consumer preferences and the willingness to adopt such a technology, contributing to economic feasibility.
- **Cost-effectiveness:** The cost of implementing hand detection technology should be weighed against the potential benefits and value it brings to the users. If the system provides a significant improvement in user experience and convenience compared to traditional volume control methods, it can be economically feasible.

3.2.3 Operational Feasibility:

- **User acceptance:** The system's success depends on user acceptance and willingness to adapt to a new control method. User feedback and usability testing can help determine the operational feasibility by assessing user satisfaction and ease of use.
- **Training and learning curve:** The feasibility study should consider the time and effort required for users to learn and adapt to using hand gestures for volume control. The system should be intuitive and user friendly to minimize the learning curve, ensuring operational feasibility.

IV. Design

4.1 Project Description:

The system allows users to adjust the volume of an audio system using hand gestures. This system utilizes computer vision techniques, including hand detection and tracking, to interpret the user's gestures and translate them into volume control commands. To implement, the system will utilize the OpenCV library for image processing and video capture, as well as pre-trained models such as the hand detection model and the hand tracking model. These models, combined with computer vision algorithms, enable accurate detection and tracking of the user's hand. The wireless volume control system by hand gestures offers a hands-free and intuitive way for users to control the audio system. It eliminates the need for physical controls and provides a seamless user experience. This project combines computer vision, audio processing, and user interface design to create a novel and interactive solution for volume control.

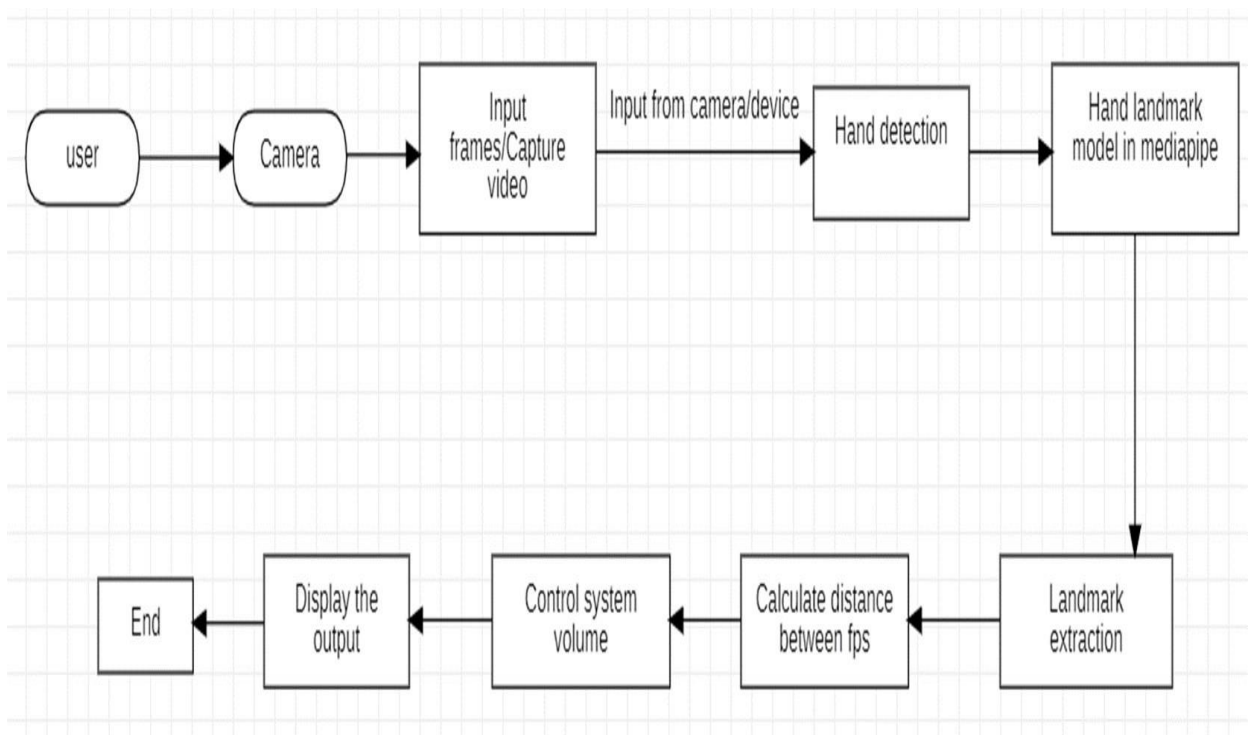


Fig. a Project Architecture

4.1.1 System Use case Diagram:

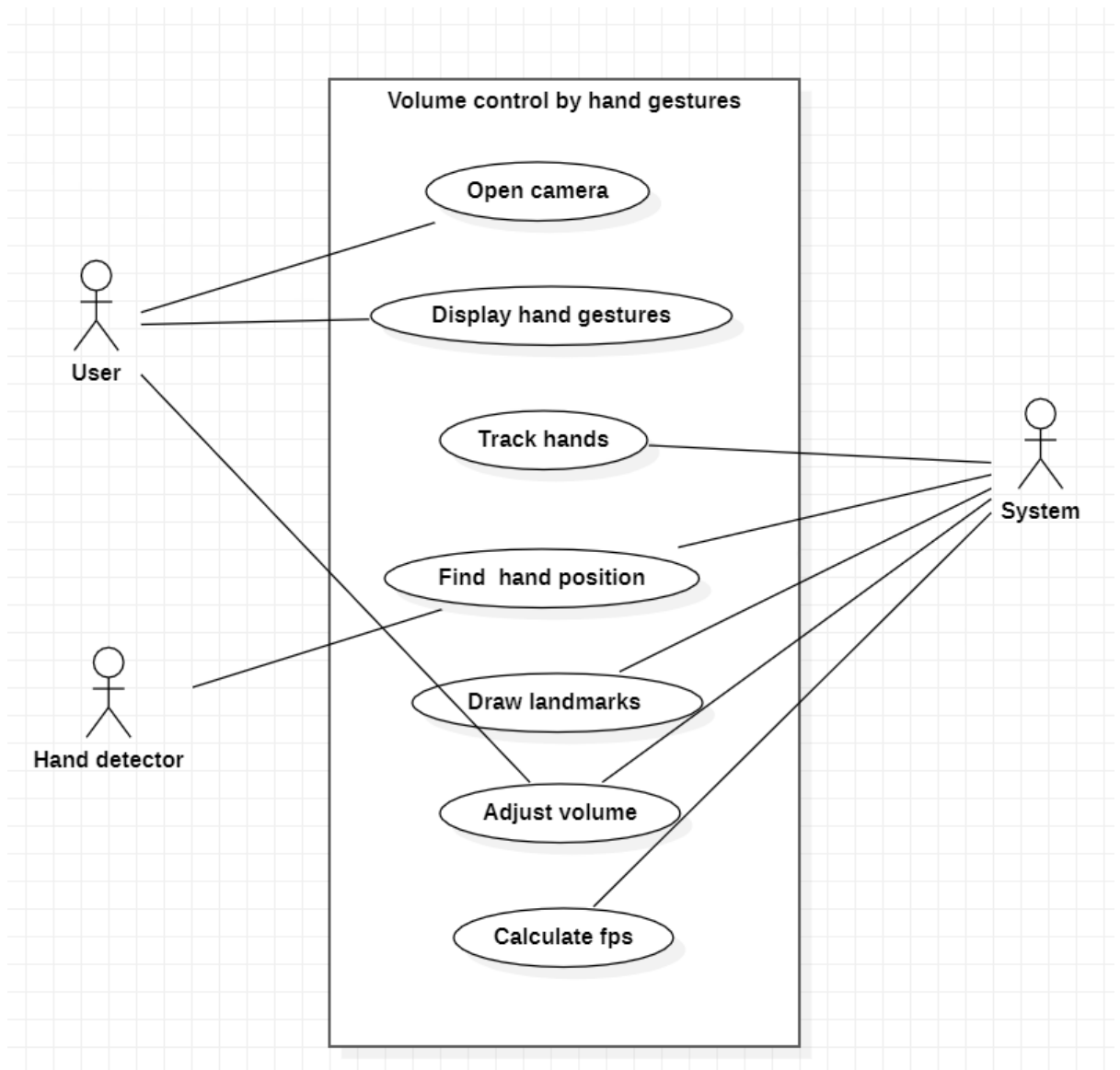


Fig. b.

The use cases in the above diagram are open camera, display hand gestures, track hands, find hand position, draw landmarks, adjust volume, calculate fps. The actors involved are user, hand detector, system. The user is given access to open camera, display hand gestures to adjust volume. The system handles the rest all use cases.

The User performs hand gestures, which are captured by the camera. The Camera captures the video stream of the user's hand gestures and sends it to the Hand Tracking Module. The Hand Tracking Module processes the video stream,

detects and tracks the position and movements of the user's hand. The Hand Landmark Model receives the tracked hand position from the Hand Tracking Module and extracts the hand landmarks or keypoints.

The Volume Control System receives the recognized hand gestures and adjusts the system volume accordingly. It interprets the gestures and determines whether to increase, decrease, or mute/unmute the audio volume.

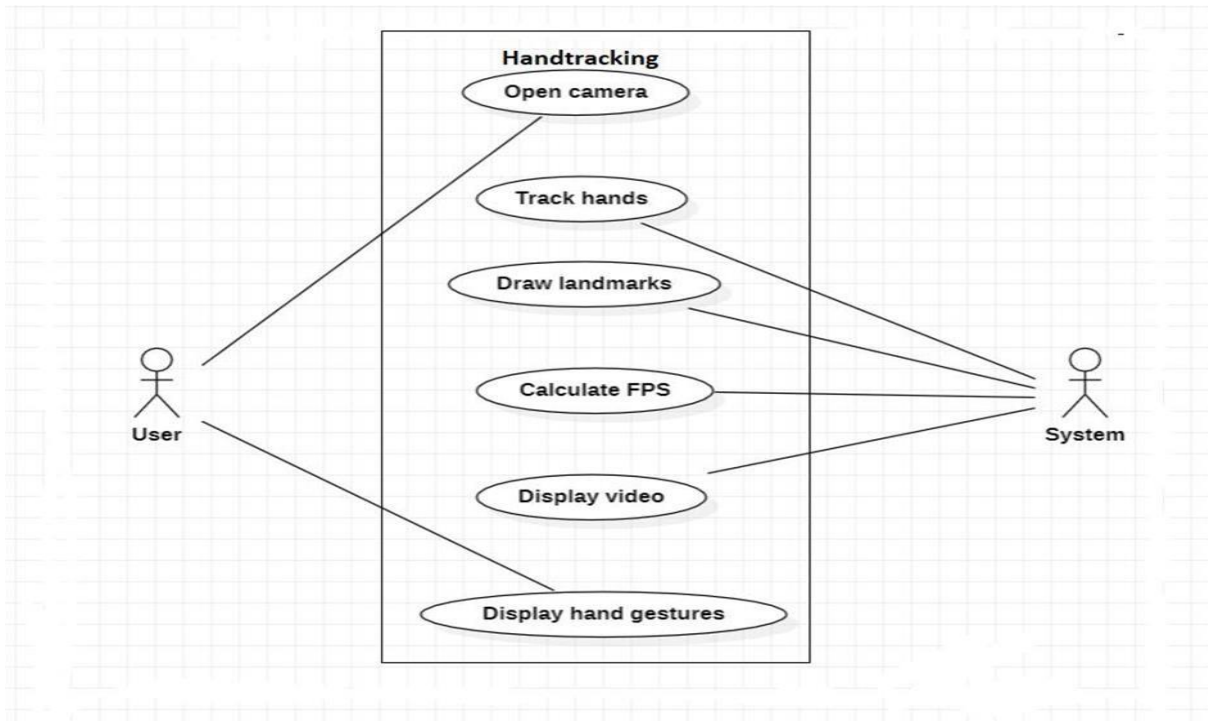


Fig c. Use-case Diagram for Hand tracking

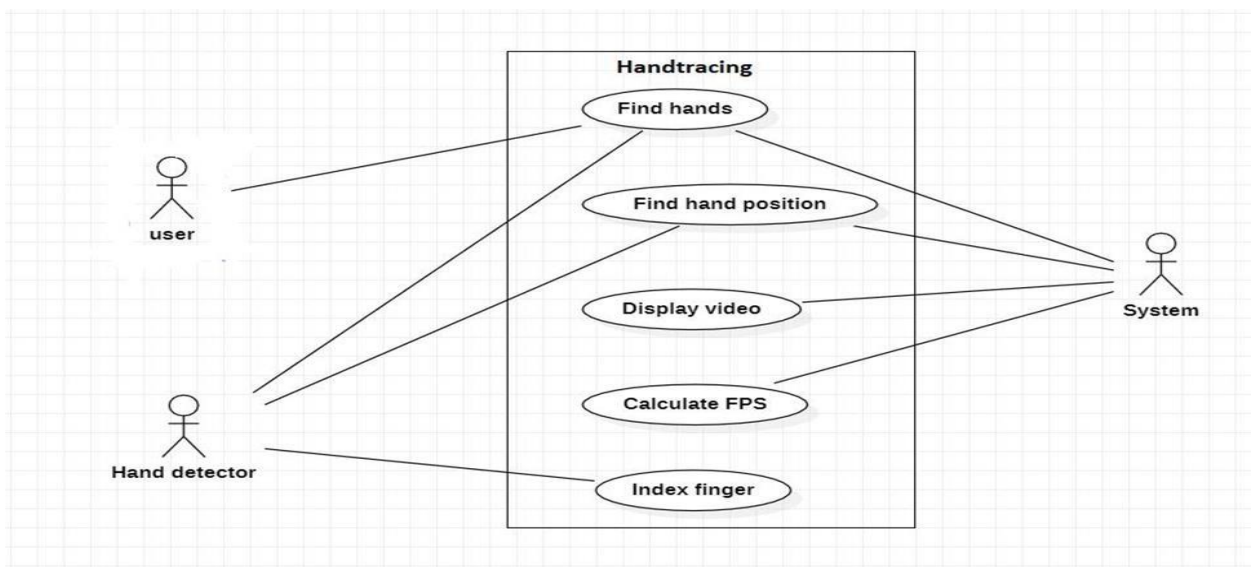


Fig. d Use-case Diagram for Hand Tracing

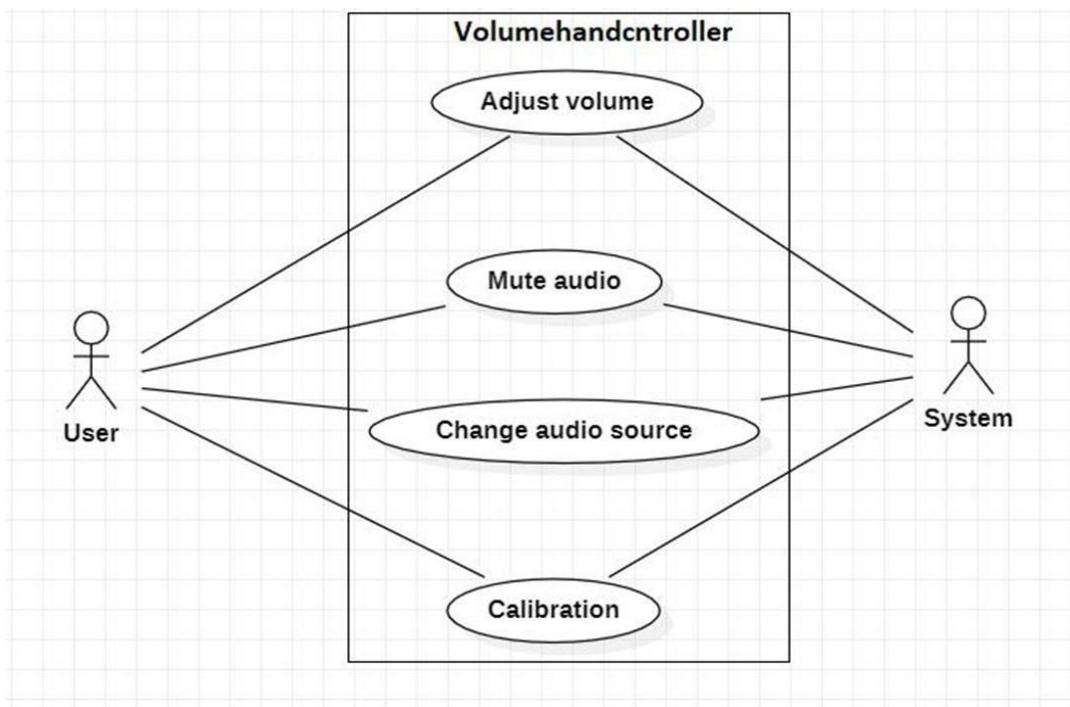


Fig. e. Use-case Diagram for Volume Controller

4.1.2 Class Diagram

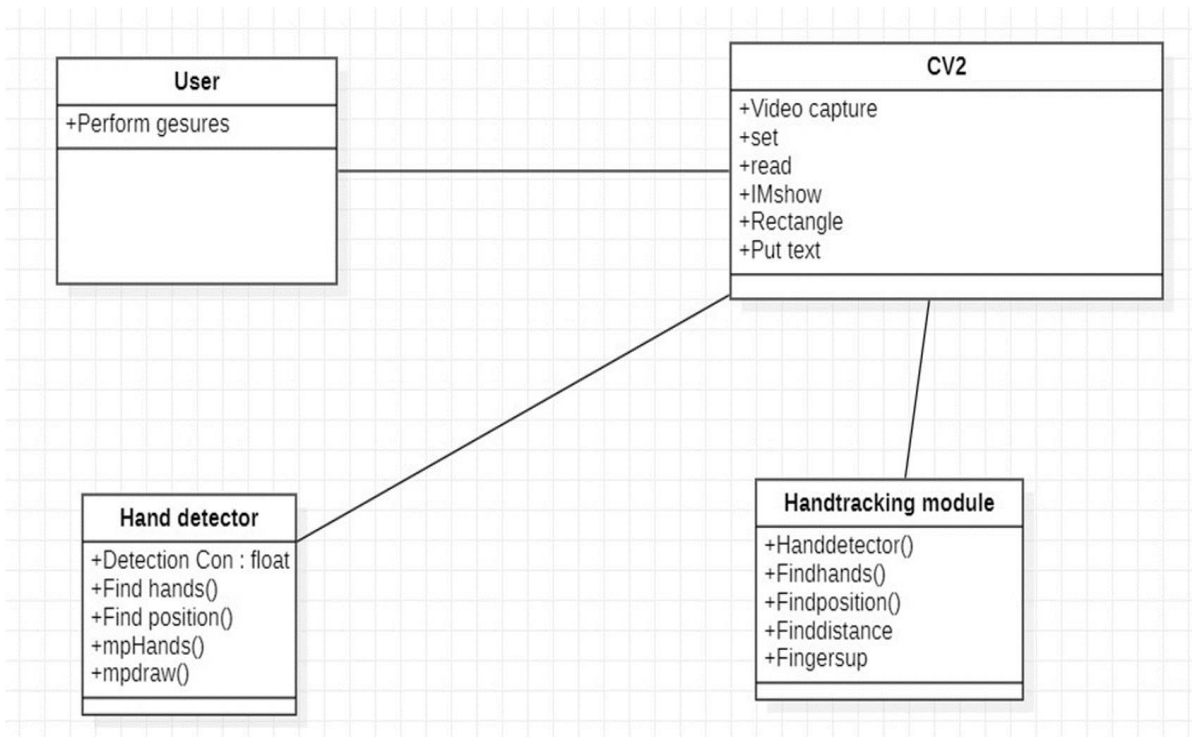


Fig. f. Class Diagram

The classes involved in the diagram are User, CV2, Hand detector and Hand tracking module.

1. User:

- Performs hand gestures.

2. CV2:

- Captures the gestures from the video
- Operations: Video capture, Set, Read, IMshow, Rectangle, Put text

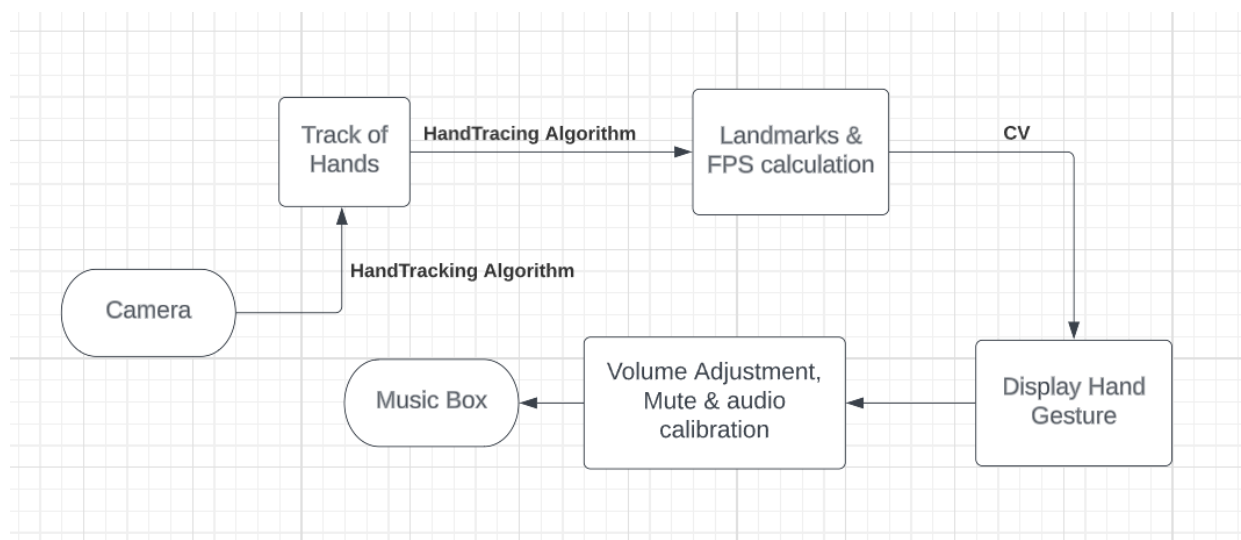
3. Hand detector:

- Identifies the presence and location of the user's hand in the video stream, enabling subsequent processing and analysis.
- Attributes: Detection Con
- Operations: Findhands() , Find position() , mpHands(), mpdraw()

4. Hand tracking:

- Identifies and tracks the landmarks of the hand, such as fingertips, palm center, and knuckles.
- Operations: Handdetector(), Findhands(), Findposition(), Finddistance(), Fingersup().

4.1.3 Data Flow Diagram (DFD):



- A data flow diagram (DFD) is a graphical representation of how data moves through a system. It illustrates the flow of data, inputs, outputs, processes, and

storage entities within a system or a specific process. For volume control using hand gestures , the dfd can help describe the process and interaction between the different components easily.

- Here the flow goes as follows the user uses the camera with the help of hand tracking module , the hands are tracked and by hand tracing algorithm the landmarks are obtained and fps is calculated. By cv module the hand gestures are displayed and with this the volume can be adjusted, muted and calibrated and the output is displayed.

V. Implementation

5.1 Source Code:

Module 1:

```
import cv2
import mediapipe as mp
import time

hand = cv2.VideoCapture(0)
uhands = mp.solutions.hands
hands = uhands.Hands()
drawing = mp.solutions.drawing_utils
prevtime = 0
currttime = 0
while True:
    success, image = hand.read()
    newimg = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    result = hands.process(newimg)
    if result.multi_hand_landmarks:
        for handLms in result.multi_hand_landmarks:
            for i, pix in enumerate(handLms.landmark):
                print(id, pix)
                h, w, c = image.shape
                cx, cy = int(pix.x * w), int(pix.y * h)
                print(id, cx, cy)
                cv2.circle(image, (cx, cy), 15, (255, 0, 255), cv2.FILLED)
            drawing.draw_landmarks(image, handLms, uhands.HAND_CONNECTIONS)
    currttime = time.time()
    frps = 1 / (currttime - prevtime)
    prevtime = currttime
    cv2.putText(image, str(int(frps)), (10, 70), cv2.FONT_HERSHEY_DUPLEX, 3, (255, 0, 255), 3)
    cv2.imshow("Image", image)
    cv2.waitKey(100)
```

Module 2:

```
import cv2
import mediapipe as mp
import time

class handDetector():
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
        self.detectionCon = detectionCon
        self.trackCon = trackCon

        self.mpHands = mp.solutions.hands
        self.hands = self.mpHands.Hands(self.mode, self.maxHands,
                                         self.detectionCon, self.trackCon)
        self.mpDraw = mp.solutions.drawing_utils

    def findHands(self, img, draw=True):
        imgofRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgofRGB)
        # print(results.multi_hand_landmarks)

        if self.results.multi_hand_landmarks:
            for handLms in self.results.multi_hand_landmarks:
                if draw:
                    self.mpDraw.draw_landmarks(img, handLms,
                                                self.mpHands.HAND_CONNECTIONS)

        return img

    def findPosition(self, img, handNo=0, draw=True):

        lmList = []
        if self.results.multi_hand_landmarks:
            myHand = self.results.multi_hand_landmarks[handNo]
            for id, lms in enumerate(myHand.landmark):
```

```

        hi, wi, ci = img.shape
        cx, cy = int(lms.x * wi), int(lms.y * hi)
        # print(id, cx, cy)
        lmList.append([id, cx, cy])
        if draw:
            cv2.circle(img, (cx, cy), 15, (255, 0, 255), cv2.FILLED)

    return lmList

def main():
    prevTime = 0
    currTime = 0
    cap = cv2.VideoCapture(0)
    detector = handDetector()
    while True:
        success, img = cap.read()
        img = detector.findHands(img)
        lmList = detector.findPosition(img)
        if len(lmList) != 0:
            print(lmList[4])

        currTime = time.time()
        fps = 1 / (currTime - prevTime)
        prevTime = currTime

        cv2.putText(img, str(int(fps)), (10, 70), cv2.FONT_HERSHEY_PLAIN, 3,
                    (255, 0, 255), 3)

        cv2.imshow("Image", img)
        cv2.waitKey(1)

if __name__ == "__main__":
    main()

```


Module 3:

```
import cv2
import time
import numpy as ns
import HandTrackingModule as htp
import math
from pyaw.pyaw import AudioUtilities, IAudioEndpointVolume
from ctypes import CLSCTX_ALL
from ctypes import cast, POINTER
widthCam, heightCam = 640, 480
capt = cv2.VideoCapture(0)
capt.set(3, widthCam)
capt.set(4, heightCam)
prevtime = 0
detector = htp.handDetector(detectionCon=0.7)
devices = AudioUtilities.GetSpeakers()
interfaces = devices.Activate(IAudioEndpointVolume.iid, CLSCTX_ALL, None)
volume = cast(interfaces, POINTER(IAudioEndpointVolume))
volume.GetMute()
#volume.GetMasterVolumeLevel()
volRange = volume.GetVolumeRange()
minVol = volRange[0]
maxVol = volRange[1]
vol = 0
volBar = 400
volPer = 0
while True:
    success, img = capt.read()
    img = detector.findHands(img)
    lmsList = detector.findPosition(img, draw=False)
    if len(lmsList) != 0:
        a1, b1 = lmsList[4][1], lmsList[4][2]
        a2, b2 = lmsList[8][1], lmsList[8][2]
        cx, cy = (a1 + a2) // 2, (b1 + b2) // 2
```

```

cv2.circle(img, (a1, b1), 15, (255, 0, 255), cv2.FILLED)
cv2.circle(img, (a2, b2), 15, (255, 0, 255), cv2.FILLED)
cv2.line(img, (a1, b1), (a2, b2), (255, 0, 255), 3)
cv2.circle(img, (cx, cy), 15, (255, 0, 255), cv2.FILLED)
lengthc = math.hypot(a2 - a1, b2 - b1)
vol = ns.interp(lengthc, [50, 300], [minVol, maxVol])
volBar = ns.interp(lengthc, [50, 300], [400, 150])
volPer = ns.interp(lengthc, [50, 300], [0, 100])
print(int(lengthc), vol)
volume.SetMasterVolumeLevel(vol, None)
if lengthc < 50:
    cv2.circle(img, (cx, cy), 15, (0, 255, 0), cv2.FILLED)
cv2.rectangle(img, (50, 150), (85, 400), (255, 0, 0), 3)
cv2.rectangle(img, (50, int(volBar)), (85, 400), (255, 0, 0), cv2.FILLED)
cv2.putText(img, f'{int(volPer)} %', (40, 450), cv2.FONT_HERSHEY_COMPLEX, 1, (255, 0, 0), 3)
currttime = time.time()
frps = 1 / (currttime - prevtime)
prevtime = currttime
cv2.putText(img, f'FPS: {int(frps)}', (40, 50), cv2.FONT_HERSHEY_COMPLEX, 1, (255, 0, 0), 3)
cv2.imshow("Img", img)
cv2.waitKey(1)

```

Module 4:

```

import cv2
import mediapipe as mp
import time
import math
class handDetector():
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
        self.detectionCon = detectionCon
        self.trackCon = trackCon

```

```

self.mpHands = mp.solutions.hands
self.hands = self.mpHands.Hands(self.mode, self.maxHands, self.detectionCon, self.trackCon)
self.tipIds = [4, 8, 12, 16, 20]
self.mpDraw = mp.solutions.drawing_utils

def find_Hands(self, img, draw=True):
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    self.results = self.hands.process(imgRGB)

    if self.results.multi_hand_landmarks:
        for handLms in self.results.multi_hand_landmarks:
            if draw:
                self.mpDraw.draw_landmarks(img, handLms, self.mpHands.HAND_CONNECTIONS)
    return img

def find_Position(self, img, handNo=0, draw=True):
    aList = []
    bList = []
    bbox = []
    self.lmList = []
    if self.results.multi_hand_landmarks:
        myHand = self.results.multi_hand_landmarks[handNo]
        for id, lm in enumerate(myHand.landmark):
            hi, wi, ci = img.shape
            cx, cy = int(lm.x * wi), int(lm.y * hi)
            aList.append(cx)
            bList.append(cy)
            self.lmList.append([id, cx, cy])
            if draw:
                cv2.circle(img, (cx, cy), 5, (255, 0, 255), cv2.FILLED)
        xmin, xmax = min(aList), max(aList)
        ymin, ymax = min(bList), max(bList)
        bbox = xmin, ymin, xmax, ymax

```

```

        if draw:
            cv2.rectangle(img, (bbox[0] - 20, bbox[1] - 20),
                           (bbox[2] + 20, bbox[3] + 20), (0, 255, 0), 2)

    return self.lmList, bbox

def fingers_Upin(self):
    fingers = []
    if self.lmcList[self.tipIds[0]][1] > self.lmcList[self.tipIds[0] - 1][1]:
        fingers.append(1)
    else:
        fingers.append(0)
    for ids in range(1, 5):
        if self.lmcList[self.tipIds[ids]][2] < self.lmcList[self.tipIds[ids] - 2][2]:
            fingers.append(1)
        else:
            fingers.append(0)
    return fingers

def find_Distancek(self, p1, p2, image, draw=True):
    h1, i1 = self.lmfList[p1][1], self.lmfList[p1][2]
    h2, i2 = self.lmfList[p2][1], self.lmfList[p2][2]
    fx, fy = (h1 + h2) // 2, (i1 + i2) // 2

    if draw:
        cv2.circle(image, (h1, i1), 15, (255, 0, 255), cv2.FILLED)
        cv2.circle(image, (h2, i2), 15, (255, 0, 255), cv2.FILLED)
        cv2.line(image, (h1, i1), (h2, i2), (255, 0, 255), 3)
        cv2.circle(image, (fx, fy), 15, (255, 0, 255), cv2.FILLED)

    lengthc = math.hypot(h2 - h1, i2 - i1)
    return lengthc, image, [h1, i1, h2, i2, fx, fy]

```

```

def main():
    prevtime = 0
    cap = cv2.VideoCapture(0)
    detector = handDetector()
    while True:
        success, img = cap.read()
        img = detector.find_Hands(img)
        lmslist, bbox = detector.find_Position(img)
        if len(lmslist) != 0:
            print(lmslist[4])

        currtime = time.time()
        frps = 1 / (currtime - prevtime)
        prevtime = currtime

        cv2.putText(img, str(int(frps)), (10, 70), cv2.FONT_HERSHEY_PLAIN, 2, (255, 0, 255), 3)
        cv2.imshow("Image", img)
        cv2.waitKey(1)

```

```

if __name__ == "__main__":
    main()

```

Module 5:

```

import cv2
import time
import numpy as np
import HandTrackingModule as htmh
from ctypes import cast, POINTER
from comtypes import CLSCTX_ALL
from pycaw.pycaw import AudioUtilities, IAudioEndpointVolume
wicam, hicam = 640, 480
cap = cv2.VideoCapture(0)

```

```

cap.set(3, wicam)
cap.set(4, hicam)
prevtime = 0
detector = htmh.handDetector(detectionCon=0.7, maxHands=1)
devices = AudioUtilities.GetSpeakers()
interface = devices.Activate(IAudioEndpointVolume.iid, CLSCTX_ALL, None)
volume = cast(interface, POINTER(IAudioEndpointVolume))
volRange = volume.GetVolumeRange()
minimumVol = volRange[0]
maximumVol = volRange[1]
vol = 0
volBar = 380
volPer = 0
areav = 0
colorVol = (255, 0, 0)

while True:
    success, img = cap.read()
    img = detector.findHands(img)
    lmlist, bobox = detector.findPosition(img, draw=True)
    if len(lmlist)!= 0:
        areav = (bobox[2] - bobox[0]) * (bobox[3] - bobox[1]) // 100
        if 250 < areav < 1000:
            leng, img, lineInfo = detector.findDistance(4, 8, img)
            volBar = np.interp(leng, [49, 201], [400, 150])
            volPer = np.interp(leng, [49, 201], [0, 100])
            smooth = 10
            volumeper = smooth * round(volPer / smooth)
            fings = detector.fingersUp()

            if not fings[4]:
                volume.SetMasterVolumeLevelScalar(volumeper / 100, None)
                cv2.circle(img, (lineInfo[4], lineInfo[5]), 15, (0, 255, 0), cv2.FILLED)
                colorVolume = (0, 255, 0)

```

```

else:
    colorVolume = (255, 0, 0)
cv2.rectangle(img, (50, 150), (85, 400), (255, 0, 0), 3)
cv2.rectangle(img, (50, int(volBar)), (85, 400), (255, 0, 0), cv2.FILLED)
cv2.putText(img, f'{int(volPer)} %', (40, 450), cv2.FONT_HERSHEY_PLAIN, 1, (255, 0, 0), 3)
currvol = int(volume.GetMasterVolumeLevelScalar() * 100)
cv2.putText(img, f'Vol Set: {int(currvol)}', (400, 50), cv2.FONT_HERSHEY_PLAIN,
            1, colorVol, 3)

currtime = time.time()
frps = 1 / (currtime - prevtime)
prevtime = currtime
cv2.putText(img, f'FPS: {int(frps)}', (40, 50), cv2.FONT_HERSHEY_PLAIN, 1, (255, 0, 0), 2)

cv2.imshow("Img", img)
cv2.imshow("Img", img)
cv2.waitKey(1)

```

MusicPlayer.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Music Player</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/5.13.0/css/all.min.css">
  <style>
    body {
      background-color: #f2f2f2;
      transition: background-color .5s;
    }
    .player {

```

```

height: 95vh;
display: flex;
align-items: center;
flex-direction: column;
justify-content: center;
}

.details {
display: flex;
align-items: center;
flex-direction: column;
justify-content: center;
margin-top: 21px;
}

.track-art {
margin: 21px;
height: 245px;
width: 245px;
background-image: url("https://images.pexels.com/photos/262034/pexels-photo-262034.jpeg?auto=compress&cs=tinysrgb&dpr=3&h=750&w=1260");
background-size: cover;
border-radius: 48%;
border: 5px solid #fff;
box-shadow: 0 0 20px rgba(0, 0, 0, 0.3);
}

.now-playing {
font-size: 1.0rem;
font-weight: bold;
margin-top: 10px;
color: #333;
}

```



```
.track-name {
  font-size: 2rem;
  font-weight: bold;
  margin-top: 10px;
  color: #333;
}

.track-artist {
  font-size: 1.2rem;
  margin-top: 5px;
  color: #666;
}

.buttons {
  display: flex;
  flex-direction: row;
  align-items: center;
  margin-top: 20px;
}

.playpause-track,
.prev-track,
.next-track {
  padding: 15px;
  opacity: 0.8;
  transition: opacity .2s;
  color: #333;
  cursor: pointer;
}

.playpause-track:hover,
.prev-track:hover,
.next-track:hover {
  opacity: 1.0;
}
```

```
}
```

```
.slider_container {  
  width: 75%;  
  max-width: 400px;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  margin-top: 30px;  
}
```

```
.seek_slider,  
.volume_slider {  
  -webkit-appearance: none;  
  -moz-appearance: none;  
  appearance: none;  
  height: 5px;  
  background: #ccc;  
  opacity: 0.7;  
  -webkit-transition: .2s;  
  transition: opacity .2s;  
}
```

```
.seek_slider::-webkit-slider-thumb,  
.volume_slider::-webkit-slider-thumb {  
  -webkit-appearance: none;  
  -moz-appearance: none;  
  appearance: none;  
  width: 15px;  
  height: 15px;  
  background: #333;  
  cursor: pointer;  
  border-radius: 50%;  
}
```

```

.seek_slider:hover,
.volume_slider:hover {
    opacity: 1.0;
}

.seek_slider {
    width: 60%;
}

.volume_slider {
    width: 30%;
}

.current-time,
.total-duration {
    padding: 10px;
    font-size: 0.8rem;
    color: #666;
}

i.fa-volume-down,
i.fa-volume-up {
    padding: 10px;
}

i.fa-play-circle,
i.fa-pause-circle,
i.fa-step-forward,
i.fa-step-backward {
    cursor: pointer;
}
</style>
</head>

```

```

<body>
  <div class="player">
    <div class="details">
      <div class="now-playing">PLAYING x OF y</div>
      <div class="track-art"></div>
      <div class="track-name">Track Name</div>
      <div class="track-artist">Track Artist</div>
    </div>
    <div class="buttons">
      <div class="prev-track" onclick="prevTrack()"><i class="fa fa-step-backward fa-2x"></i></div>
      <div class="playpause-track" onclick="playpauseTrack()"><i class="fa fa-play-circle fa-
5x"></i></div>
      <div class="next-track" onclick="nextTrack()"><i class="fa fa-step-forward fa-2x"></i></div>
    </div>
    <div class="slider_container">
      <div class="current-time">00:00</div>
      <input type="range" min="1" max="100" value="0" class="seek_slider" onchange="seekTo()">
      <div class="total-duration">00:00</div>
    </div>
  </div>
  <!-- Main script for the player -->
  <script>
    let nowPlaying = document.querySelector(".now-playing");
    let trackArt = document.querySelector(".track-art");
    let trackName = document.querySelector(".track-name");
    let trackArtist = document.querySelector(".track-artist");

    let playpauseBtn = document.querySelector(".playpause-track");
    let nextBtn = document.querySelector(".next-track");
    let prevBtn = document.querySelector(".prev-track");

    let seekSlider = document.querySelector(".seek_slider");
    let volumeSlider = document.querySelector(".volume_slider");
  </script>

```

```

let currTime = document.querySelector(".current-time");
let totalDuration = document.querySelector(".total-duration");
let trackIndex = 0;
let isPlaying = false;
let updateTimer;
let currTrack = document.createElement('audio');
let trackList = [
  {
    name: "Nature",
    artist: "McJones",
    image: "https://images.pexels.com/photos/2264753/pexels-photo-2264753.jpeg?auto=compress&cs=tinysrgb&dpr=3&h=250&w=250",
    path: "https://files.freemusicarchive.org/storage-freemusicarchive-org/music/WFMU/Broke_For_Free/Directionless_EP/Broke_For_Free_-01-_Night_Owl.mp3"
  },
  {
    name: "Peace",
    artist: "Jimin",
    image: "https://images.pexels.com/photos/3100835/pexels-photo-3100835.jpeg?auto=compress&cs=tinysrgb&dpr=3&h=250&w=250",
    path: "https://files.freemusicarchive.org/storage-freemusicarchive-org/music/no_curator/Tours/Enthusiast/Tours_-01-_Enthusiast.mp3"
  },
  {
    name: "Lanes",
    artist: "JimmyFallon",
    image: "https://images.pexels.com/photos/1717969/pexels-photo-1717969.jpeg?auto=compress&cs=tinysrgb&dpr=3&h=250&w=250",
    path: "https://files.freemusicarchive.org/storage-freemusicarchive-org/music/ccCommunity/Chad_Crouch/Arps/Chad_Crouch_-_Shipping_Lanes.mp3",
  },
];

function loadTrack(trackIndex) {

```

```

clearInterval(updateTimer);
resetValues();
currTrack.src = trackList[trackIndex].path;
currTrack.load();
trackArt.style.backgroundImage = "url(" + trackList[trackIndex].image + ")";
trackName.textContent = trackList[trackIndex].name;
trackArtist.textContent = trackList[trackIndex].artist;
nowPlaying.textContent = "PLAYING " + (trackIndex + 1) + " OF " + trackList.length;
updateTimer = setInterval(seekUpdate, 1000);
currTrack.addEventListener("ended", nextTrack);
randomBgColor();
}

```

```

function randomBgColor() {
  let red = Math.floor(Math.random() * 256) + 64;
  let green = Math.floor(Math.random() * 256) + 64;
  let blue = Math.floor(Math.random() * 256) + 64;
  let bgColor = "rgb(" + red + "," + green + "," + blue + ")";
  document.body.style.background = bgColor;
}

```

```

function resetValues() {
  currTime.textContent = "00:00";
  totalDuration.textContent = "00:00";
  seekSlider.value = 0;
}

```

```

function playpauseTrack() {
  if (!isPlaying) playTrack();
  else pauseTrack();
}

```

```

function playTrack() {
  currTrack.play();
}

```

```

    isPlaying = true;
    playpauseBtn.innerHTML = '<i class="fa fa-pause-circle fa-5x"></i>';
}

function pauseTrack() {
    currTrack.pause();
    isPlaying = false;
    playpauseBtn.innerHTML = '<i class="fa fa-play-circle fa-5x"></i>';
}

function nextTrack() {
    if (trackIndex < trackList.length - 1)
        trackIndex += 1;
    else trackIndex = 0;
    loadTrack(trackIndex);
    playTrack();
}

function prevTrack() {
    if (trackIndex > 0)
        trackIndex -= 1;
    else trackIndex = trackList.length;
    loadTrack(trackIndex);
    playTrack();
}

function seekTo() {
    seekto = currTrack.duration * (seekSlider.value / 100);
    currTrack.currentTime = seekto;
}

function setVolume() {
    currTrack.volume = volumeSlider.value / 100;
}

```

```

function seekUpdate() {
  let seekPosition = 0;
  if (!isNaN(currTrack.duration)) {
    seekPosition = currTrack.currentTime * (100 / currTrack.duration);
    seekSlider.value = seekPosition;
    let currentMinutes = Math.floor(currTrack.currentTime / 60);
    let currentSeconds = Math.floor(currTrack.currentTime - currentMinutes * 60);
    let durationMinutes = Math.floor(currTrack.duration / 60);
    let durationSeconds = Math.floor(currTrack.duration - durationMinutes * 60);
    if (currentSeconds < 10) { currentSeconds = "0" + currentSeconds; }
    if (durationSeconds < 10) { durationSeconds = "0" + durationSeconds; }
    if (currentMinutes < 10) { currentMinutes = "0" + currentMinutes; }
    if (durationMinutes < 10) { durationMinutes = "0" + durationMinutes; }
    currTime.textContent = currentMinutes + ":" + currentSeconds;
    totalDuration.textContent = durationMinutes + ":" + durationSeconds;
  }
}

loadTrack(trackIndex);
</script>
</body>

</html>

```


5.2 Results:

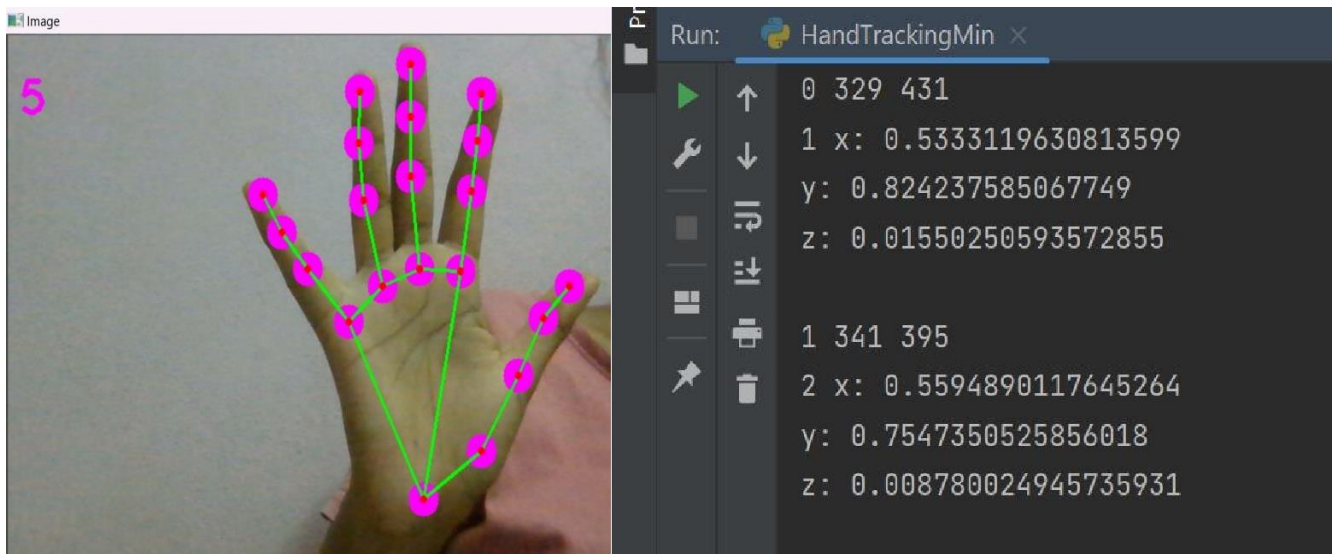


Fig. x.

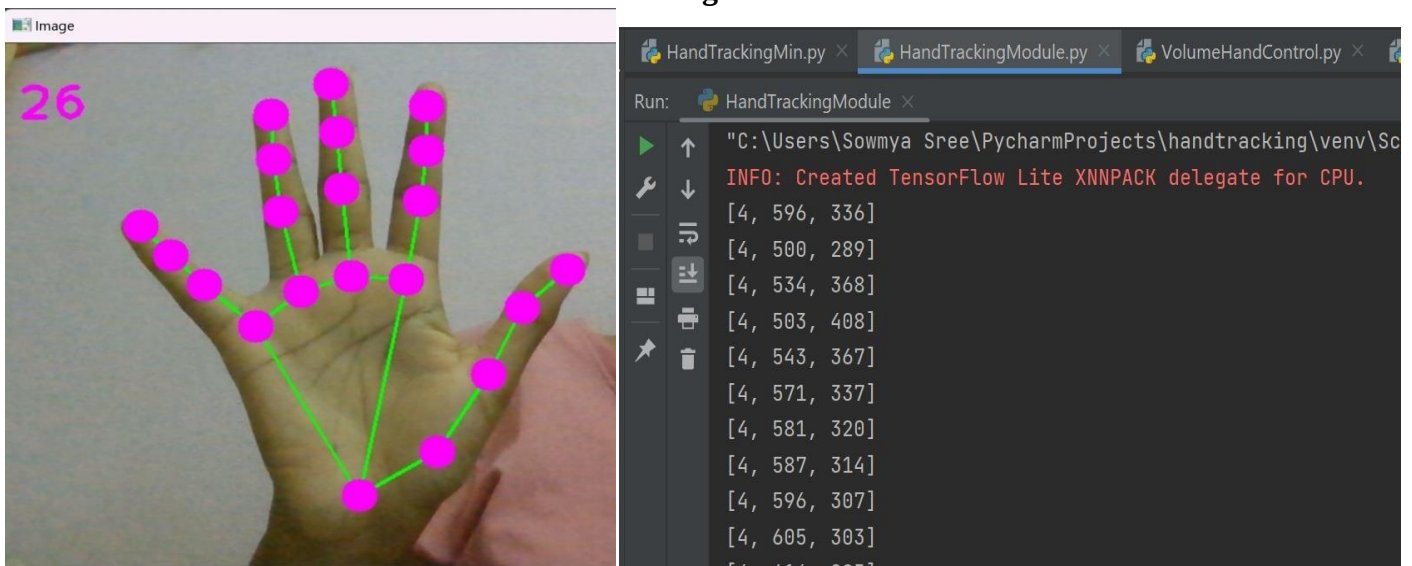


Fig. x.

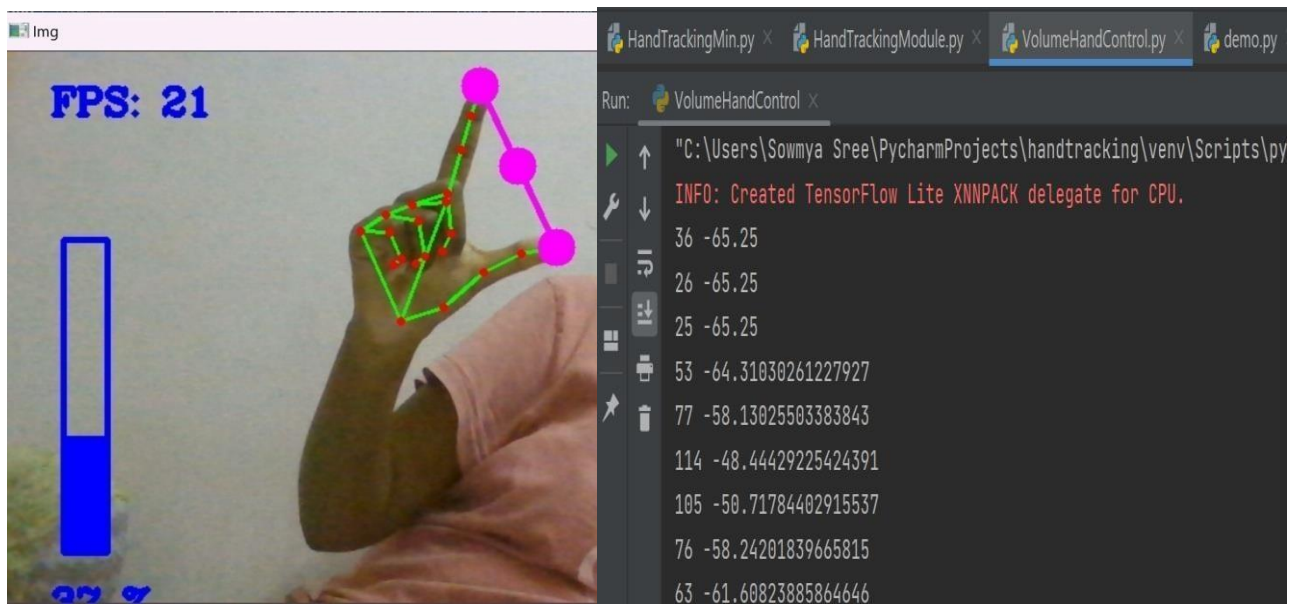


Fig. x.

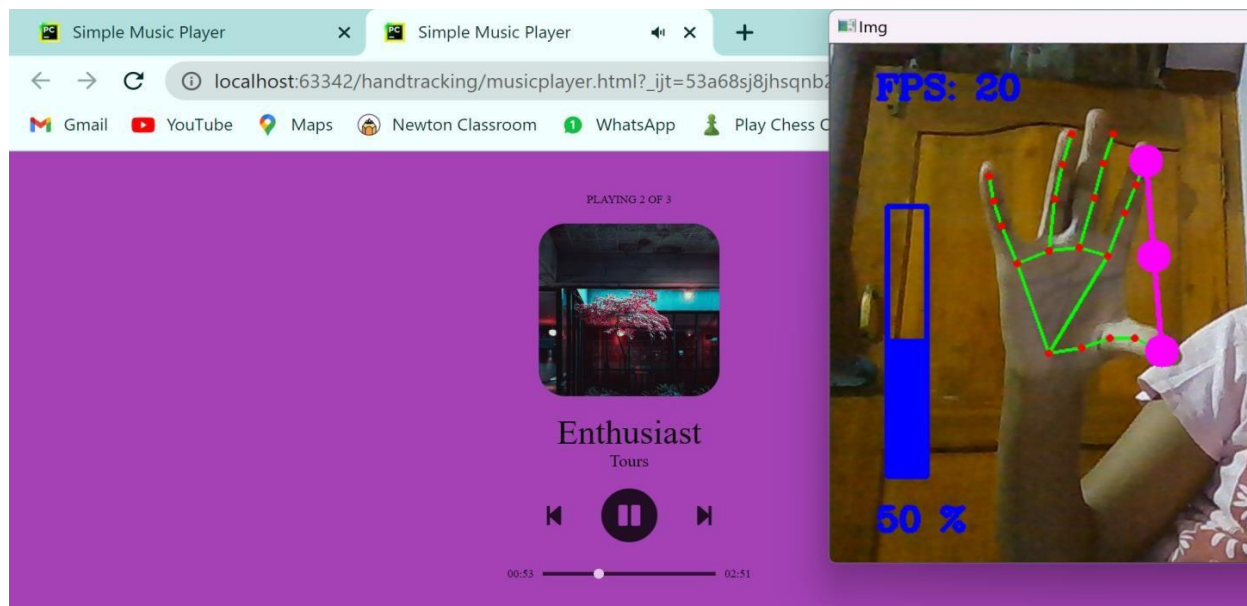


Fig. X.

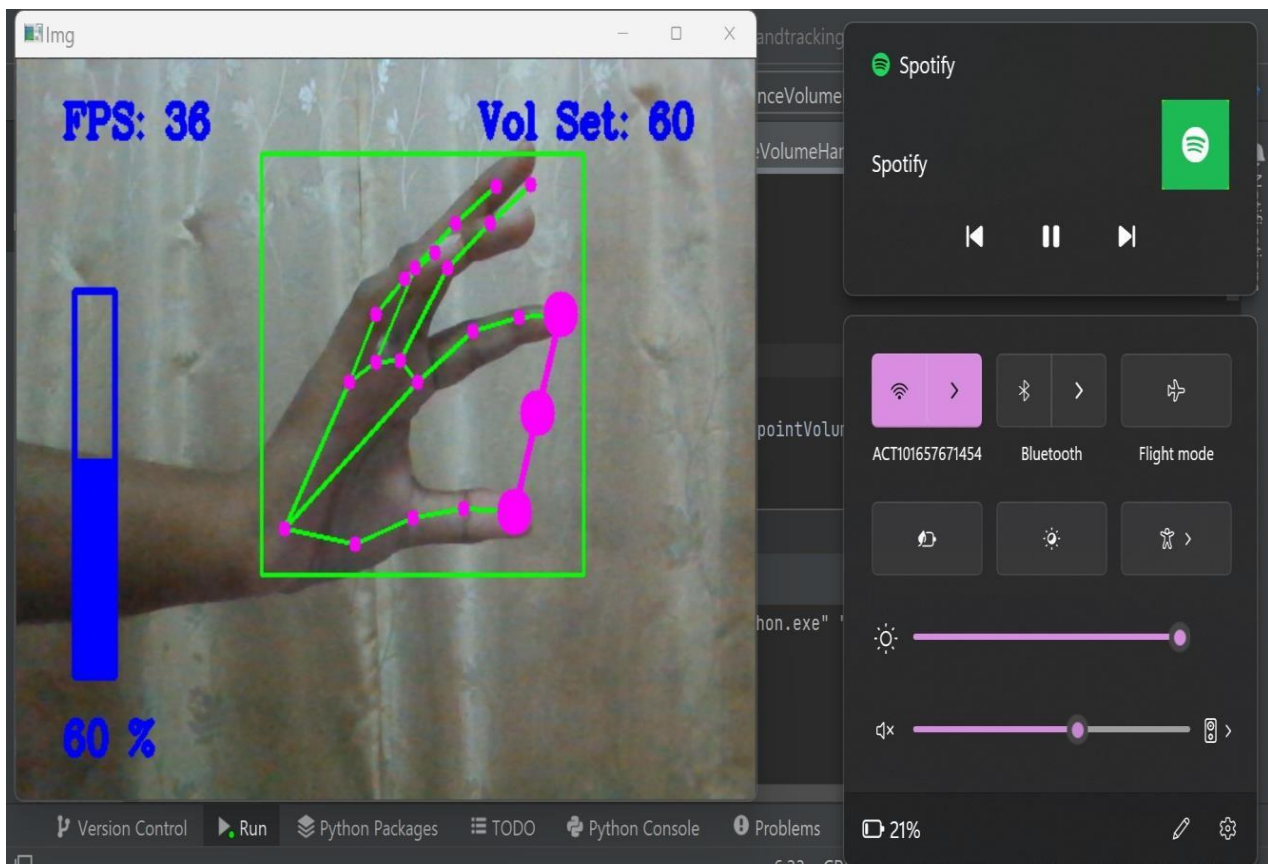
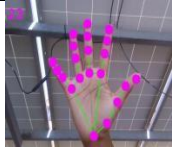
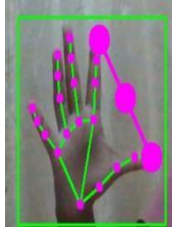
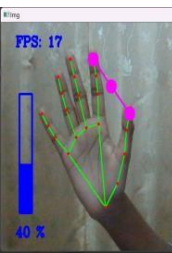
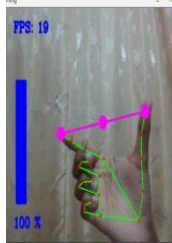
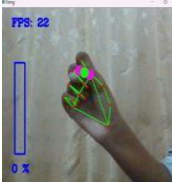
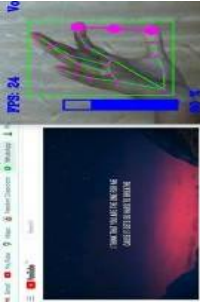


Fig. x.

Test Case ID	Test Scenario	Pre-conditions	Test steps	Test Data	ExpectedResults	Actual Results	Status Pass/Fail
1.	Hand tracking	Execution of program	Select the RUN option and then place your palm in front of the webcam		The hand should be detected which is highlighted by landmarks in output.	The hand has been detected by landmarks on it.	Pass
2.	Hand tracking by gestures	Execution of program	Select the RUN option and then place your palm in front of the webcam		The hand should be tracked by detecting the index finger and fps.	The hand is tracked and index finger presence is determined in finger print landmarks on user hand.	Pass
3.	Volume control	Execution of program	Select the RUN option and then place your palm in front of the webcam and perform gestures by using index and thumb finger.		The hand gestures should be tracked and accordingly volume should increase or decrease of the system.	The system volume is controlled according to the user hand gestures performed.	Pass
4.	Increase volume to maximum	Execution of program	Select the RUN option and then place your palm in front of the webcam and perform gestures by using index and thumb finger by moving them apart.		The system volume should increase on tracking gestures through webcam.	The system volume has increased to maximum level.	Pass
5.	Mute command using Volume controller	Execution of program	Select the RUN option and then place your palm in front of the webcam. Close the palm.		The system volume should be minimum or mute.	The system volume is decreased to minimum level.	Pass
6.	Using volume controller on local audio files	Execution of program	Select a MP3 file and play it. Select the RUN option and then place your palm in front of the webcam and perform gestures using index and thumb fingers.		The audio file playing in the background must be audible according to the volume controlling.	The audio playing in system is well controlled by the volume controller according to user choice.	Pass

7.	Using volume control on online audio files	Execution of program	Select an audio file through internet and play. Select the RUN option and then place your palm in front of the webcam and perform gestures using index and thumb fingers.		The volume should be controlled or audible according to gestures.	This volume controller also performs on the online audio files.	Pass
8.	Using Volume controller on online videos.	Execution of program	Select a video from internet and play it. Select the RUN option and then place your palm in front of the webcam and perform gestures using index and thumb fingers.		The corresponding video's audio should be controlled according to user.	This even works on online videos playing in the system	Pass
9.	Volume controller on customized music playlist	Execution of program	Open music player page, select music and play it. Select the RUN option and then place your palm in front of the webcam and perform gestures using index and thumb fingers.		The songs in the playlist should be audible according to user necessity.	The songs are played on page created and volume is also controlled	Pass
10.	To check if system volume is functioning in parallel or not	Execution of program	Select an mp3 file and play .Select the RUN option and then place your palm in front of the webcam and perform gestures using index and thumb fingers.		The system should the parallel increase and decrease influence on volume.	The volume controller has parallely displayed and controlled the system's volume.	Pass

VII. Conclusion & Future Scope:

- **Conclusion**

In conclusion, the proposed gesture recognition system offers an innovative approach to bridge the gap between humans and machines by utilizing hand gestures for volume control. By employing computer vision techniques and deep learning algorithms such as CNN+LSTM, the system effectively captures and interprets the motions of the human body through a computer camera. The development of the system took place using PyCharm as the IDE and Python as the programming language. OpenCV version 3.x was utilized to access a wide range of image and video processing functions. The system focuses on detecting the index and thumb fingers, calculating the distance between them, and adjusting the volume accordingly. A visual volume bar is displayed in the top right corner to provide users with real-time feedback on the volume level.

The integration of computer vision and deep learning algorithms enables the system to achieve efficient and real-time volume control based on hand gestures. This novel interaction method enhances user experience by offering a more natural and intuitive way to interact with applications and devices. The successful implementation of the proposed system paves the way for broader applications in the field of human-computer interaction. Future research can explore expanding the range of recognized hand gestures and integrating additional functionalities to create a comprehensive gesture-based control system.

In summary, this system demonstrates the potential of utilizing hand gestures and computer vision techniques for volume control. By effectively capturing and interpreting hand motions, the system establishes a more potent and intuitive link between humans and machines, offering enhanced user experiences in real-time applications.

- **Future Scope**

The proposed gesture recognition system for volume control through hand gestures opens up several avenues for future research and development. Some potential areas of future exploration and enhancement include:

1. **Expanded Gesture Recognition:** The system can be further developed to recognize and interpret a wider range of hand gestures beyond just controlling volume. By expanding the repertoire of recognized gestures, users can interact with a broader range of applications and functionalities, making the system more versatile and user-friendly.

2. **Integration with Multiple Devices:** Currently, the system focuses on controlling volume through hand gestures. However, in the future, it can be extended to incorporate control over other devices and applications, such as multimedia players, smart home systems, or virtual reality environments. This expansion would allow users to interact with a wide array of devices using intuitive hand gestures.

3. **Enhanced Accuracy and Robustness:** Ongoing research can be directed towards improving the accuracy and robustness of hand gesture recognition. Advanced computer vision techniques, machine learning algorithms, and larger datasets can be utilized to enhance the system's ability to precisely detect and interpret hand gestures, even in challenging environments or with varying lighting conditions.

4. **Multi-modal Interaction:** The system can be extended to support multi-modal interaction, combining hand gestures with other input modalities such as voice recognition or facial expressions. This integration would provide users with a more immersive and seamless interaction experience, enabling them to control devices and applications through a combination of gestures, voice commands, and other cues.

5. **Application in Assistive Technologies:** Gesture recognition systems have the potential to greatly benefit individuals with physical disabilities. Future research can explore the application of this system in assistive technologies, such as controlling devices for people with limited mobility. Adapting the system to cater to specific accessibility needs can greatly enhance the quality of life for individuals with disabilities.

6. Real-time Gesture Training: A future direction could involve developing a real-time gesture training module that allows users to create their own custom gestures for specific actions or commands. This would enable users to personalize their interaction with devices and applications based on their preferences and needs.

In summary, the proposed gesture recognition system for volume control lays the foundation for further advancements in human-computer interaction. Future research can focus on expanding gesture recognition capabilities, integrating with multiple devices, improving accuracy and robustness, exploring multi-modal interaction, applying the system in assistive technologies, and enabling real-time gesture training. These advancements will contribute to creating more intuitive and immersive user experiences in various domains.

VIII. References (Papers, Websites, Textbooks):

Papers:

- [1]<https://ieeexplore.ieee.org/abstract/document/7150886/> Real time finger tracking and contour detection for gesture recognition using OpenCV.
- [2]<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.667.104&rep=rep1&type=pdf> Hand Gesture Recognition Techniques For Human Computer Interaction.
- [3]https://link.springer.com/chapter/10.1007/978-3-030-41862-5_174 Hand gesture recognition using OpenCv and Python
- [4][https://ijisrt.com/assets/upload/files/IJISRT22MAY250_\(1\).pdf](https://ijisrt.com/assets/upload/files/IJISRT22MAY250_(1).pdf) Volume Control using Gestures.

